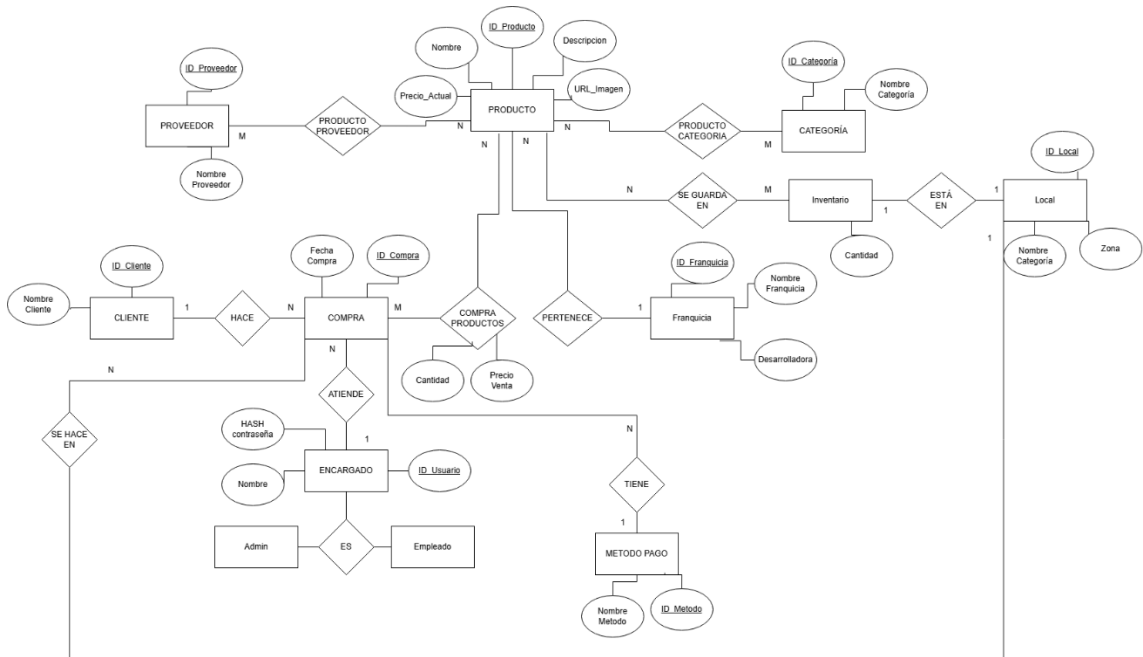
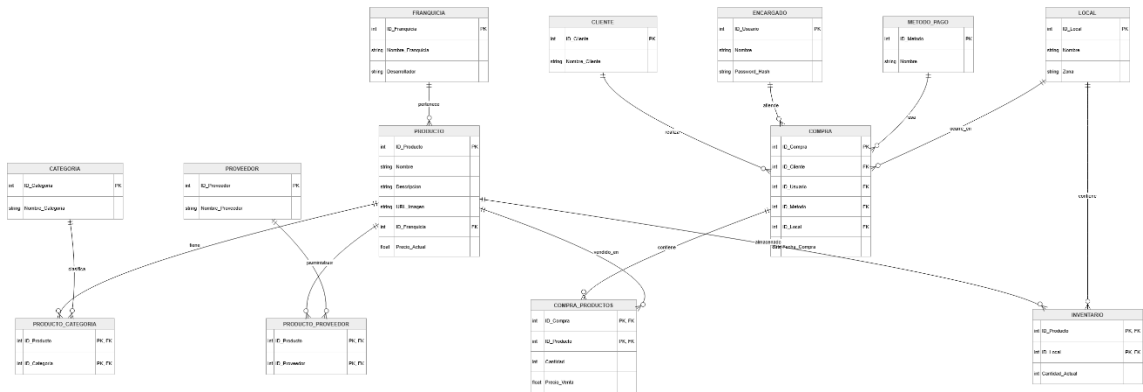


Proyecto 2

Diagrama Entidad Relación



Esquema en Notación Relacional



PRODUCTO

ID_Producto, Nombre, Descripcion, URL_Imagen, ID_Franquicia, Precio_Actual

PK: ID_Producto

FK: ID_Franquicia → FRANQUICIA(ID_Franquicia)

CATEGORIA

ID_Categoria, Nombre_Categoria

PK: ID_Categoria

PRODUCTO_CATEGORIA

ID_Producto, ID_Categoria

PK: (ID_Producto, ID_Categoria)

FK: ID_Producto → PRODUCTO(ID_Producto)

FK: ID_Categoria → CATEGORIA(ID_Categoria)

FRANQUICIA

ID_Franquicia, Nombre_Franquicia, Desarrollador

PK: ID_Franquicia

PROVEEDOR

ID_Proveedor, Nombre_Proveedor

PK: ID_Proveedor

PRODUCTO_PROVEEDOR

ID_Producto, ID_Proveedor

PK: (ID_Producto, ID_Proveedor)

FK: ID_Producto → PRODUCTO(ID_Producto)

FK: ID_Proveedor → PROVEEDOR(ID_Proveedor)

CLIENTE

ID_Cliente, Nombre_Cliente

PK: ID_Cliente

EMPLEADO

ID_Empleado, Nombre_Empleado

PK: ID_Empleado

METODO_PAGO

ID_Metodo, Nombre

PK: ID_Metodo

LOCAL

ID_Local, Nombre, Zona

PK: ID_Local

COMPRA

ID_Compra, ID_Cliente, ID_Empleado, ID_Metodo, ID_Local, Fecha_Compra

PK: ID_Compra

FK: ID_Cliente → CLIENTE(ID_Cliente)

FK: ID_Empleado → EMPLEADO(ID_Empleado)

FK: ID_Metodo → METODO_PAGO(ID_Metodo)

FK: ID_Local → LOCAL(ID_Local)

COMPRA_PRODUCTOS

ID_Compra, ID_Producto, Cantidad, Precio_Venta

PK: (ID_Compra, ID_Producto)

FK: ID_Compra → COMPRA(ID_Compra)

FK: ID_Producto → PRODUCTO(ID_Producto)

INVENTARIO

ID_Producto, ID_Local, Cantidad_Actual

PK: (ID_Producto, ID_Local)

FK: ID_Producto → PRODUCTO(ID_Producto)

FK: ID_Local → LOCAL(ID_Local)

Normalización Justificada

Para el diseño de la base de datos se aplicó normalización con el objetivo de evitar redundancias y problemas al momento de insertar, actualizar o eliminar datos.

Se partió del modelo ER y luego se pasó a un modelo relacional, asegurando que todas las tablas cumplan hasta la tercera forma normal.

Primera Forma Normal

Datos atómicos, todas las tablas cumplen con esto porque cada atributo guarda un solo valor, por ejemplo: Nombre, Precio_Actual y Cantidad.

Y las relaciones muchos a muchos se separaron en tablas independientes:

- PRODUCTO_CATEGORIA
- PRODUCTO_PROVEEDOR
- COMPRA_PRODUCTOS

Esto hace que no haya una lista de datos dentro de una misma tabla

Segunda Forma Normal

No deben haber atributos con dependencias parciales, esto se cumple con todas las tablas compuestas del proyecto, aquí hay algunas:

- COMPRA_PRODUCTOS
 - PK: (ID_Compra, ID_Producto)
 - Atributos: Cantidad, Precio_Venta

En este caso Cantidad y Precio_Venta dependen de la combinación entera y no solo de ID_Compra
o ID_Producto

- INVENTARIO

- PK: (ID_Producto, ID_Local)
- Atributos: Cantidad_Actual

Para este Cantidad_Actual depende del producto como del local a la vez, el stock cambia según la tienda

Tercera Forma Normal

No deben haber dependencias transitivas.

Aquí voy a poner un ejemplo de cómo NO se debe hacer:

Si la tabla Producto se hubiese hecho así:

PRODUCTO (ID_Producto, Nombre, ID_Franquicia, Nombre_Franquicia, Desarrollador)

Empezaría a existir una dependencia transitiva al tener la info de la franquicia dentro del producto

ID_Producto → ID_Franquicia → Nombre_Franquicia, Desarrollador

Entonces para evitar que sucediese esto únicamente se separó en otra tabla:

FRANQUICIA (ID_Franquicia → Nombre_Franquicia, Desarrollador)

Y producto solo guarda la referencia de la franquicia como una FK.

Tabla COMPRA

$ID_Compra \rightarrow ID_Cliente, ID_Empleado, ID_Metodo, ID_Local, Fecha_Compra$

Todos los atributos de la tabla compra dependen de la PK

No hay atributos que dependan de otros atributos clave.

Tabla METODO_PAGO y LOCAL

$ID_Metodo \rightarrow Nombre$

$ID_Local \rightarrow Nombre, Zona$

Dependencias simples, no hay relaciones internas entre los atributos.

REPO GITHUB:

<https://github.com/SrCharlied/24531-GameStore>